

Introdução à Programação C

Aula 04 - Ponteiros, Vetores e Alocação Dinâmica

Gustavo Henrique Aragão Silva



Quem sou eu?

- Estudante de **Ciência da Computação** (8º Período)
- **Ex-Monitor** de Programação Imperativa
- **Desenvolvedor Back-end** na Di2Win
- **Presidente** da **Liga Acadêmica de Desenvolvimento Web (LAWD)**
- Viciado em **Programação Competitiva** 🎈
 - **Finalista** da Maratona SBC 2025 e **Campeão** da ProgBase 2025



GitHub: @gustaaragao



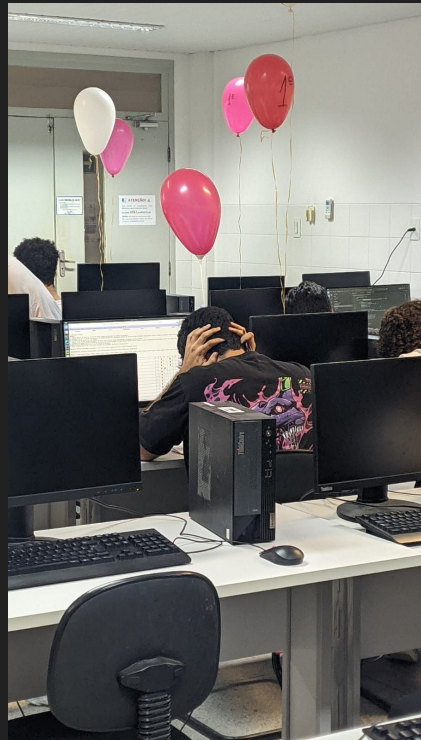
Linkedin: gustavoaragao



E-mail: gustavo.aragao@dcomp.ufs.br



Discord: gustaaragao



Ponteiros



Obrigado, Calumbi pelos
slides e desenhos

Como funciona a memória de um computador?

Memória Ram
(fictícia)

H000	####
...	####
S008	####
S004	####
S000	####

Vamos analisar esse código para entender



```
int a = 10;  
int b, c;
```

```
b = 20;  
c = a + b;
```

Memória Ram
(fictícia)

H000	####
...	####
S008	####
S004	####
S000	####

Vamos analisar esse código para entender



```
int a = 10;  
int b, c;  
  
b = 20;  
c = a + b;
```



Memória Ram
(fictícia)

H000	####
...	####
S008	####
S004	####
S000	10

a



```
&a = S000;  
a = 10;
```



Vamos analisar esse código para entender



```
int a = 10;  
int b, c;  
  
b = 20;  
c = a + b;
```



Memória Ram
(fictícia)

H000	####	
...	####	
S008	####	c
S004	####	b
S000	10	a



```
&a = S000; a = 10;  
&b = S004; b = ####;  
&c = S008; c = ####;
```


Vamos analisar esse código para entender



```
int a = 10;  
int b, c;  
  
b = 20;  
c = a + b;
```



Memória Ram
(fictícia)

H000	####	
...	####	
S008	####	c
S004	####	b
S000	10	a



```
&a = S000; a = 10;  
&b = S004; b = ####;  
&c = S008; c = ####;
```

Vamos analisar esse código para entender



```
int a = 10;  
int b, c;  
  
b = 20;  
c = a + b;
```

Memória Ram
(fictícia)

H000	####	
...	####	
S008	####	c
S004	20	b
S000	10	a

```
&a = S000; a = 10;  
&b = S004; b = 20;  
&c = S008; c = ####;
```

Vamos analisar esse código para entender



```
int a = 10;  
int b, c;  
  
b = 20;  
c = a + b;
```

Memória Ram
(fictícia)

H000	####	
...	####	
S008	30	c
S004	20	b
S000	10	a



&a = S000; a = 10;
&b = S004; b = 20;
&c = S008; c = 30



```
#include <stdio.h>
```

```
int main(void)
{
```

```
    int a = 10;
    int b, c;
```

```
    printf("&a = %p; a = %d\n", &a,a);
    printf("&b = %p; b = %d\n", &b,b);
    printf("&c = %p; c = %d\n\n", &c,c);
```

```
    b = 20;
    c = a + b;
```

```
    printf("&a = %p; a = %d\n", &a,a);
    printf("&b = %p; b = %d\n", &b,b);
    printf("&c = %p; c = %d\n\n", &c,c);
```

```
    return 0;
```

```
}
```

Exemplo real:

Execução no terminal

```
&a = 0x7ffdc1e00a2c; a = 10
&b = 0x7ffdc1e00a28; b = -1042281640
&c = 0x7ffdc1e00a24; c = 0
```

```
&a = 0x7ffdc1e00a2c; a = 10
&b = 0x7ffdc1e00a28; b = 20
&c = 0x7ffdc1e00a24; c = 30
```

Ponteiros



Variáveis que apontam para um endereço de memória

O que é um ponteiro?

- É uma variável que armazena o endereço de memória de outra variável.
- Permite acessar e manipular dados armazenados em memória.
- Facilita operações com arrays.
- Permite alterar valores originais fora do escopo da função.

Como declarar um ponteiro?



```
#include <stdio.h>

int main(void)
{
    int a = 10;

    int* p1 = &a;
    int* p2 = NULL;
    int* p3;

    return 0;
}
```

Como manipular um ponteiro?

Vamos entender o que
está acontecendo aqui



```
#include <stdio.h>

int main(void)
{
    int a = 10;

    int* p1 = &a;
    int* p2 = NULL;

    p2 = p1;
    *p2 = 20;

    return 0;
}
```




```
int a = 10;  
  
int* p1 = &a;  
int* p2 = NULL;  
  
p2 = p1;  
*p2 = 20;
```

Memória Ram (fictícia)

H000

####

...

####

S012

####

S004

####

S000

####

Memória Ram (fictícia)

H000

####

...

####

S012

####

S004

####

S000

10

a

`&a = S000; a = 10;`



```
int a = 10;
```

```
int* p1 = &a;  
int* p2 = NULL;
```

```
p2 = p1;  
*p2 = 20;
```



```
int a = 10;
```

```
int* p1 = &a;
```

```
int* p2 = NULL;
```

```
p2 = p1;
```

```
*p2 = 20;
```

Memória Ram (fictícia)

H000	####
...	####
S012	####
S004	S000
S000	10



p1

a

```
&a = S000; a = 10;  
&p1 = S004; p1 = S000; *p1 = 10;
```

**** dizemos que o ponteiro 'p1' aponta pra S000**

Memória Ram
(fictícia)



```
int a = 10;  
  
int* p1 = &a;  
int* p2 = NULL;  
  
p2 = p1;  
*p2 = 20;
```



H000

####

...

####

S012

####

S004

S000

S000

10



p1

a

&a = S000; a = 10;
&p1 = S004; p1 = S000; *p1 = 10;



```
int a = 10;
```

```
int* p1 = &a;
```

```
int* p2 = NULL;
```

```
p2 = p1;
```

```
*p2 = 20;
```



Memória Ram (fictícia)

H000	####	
...	####	
S012	NULL	p2
S004	S000	p1
S000	10	a



```
&a = S000; a = 10;  
&p1 = S004; p1 = S000; *p1 = 10;  
&p2 = S012; p2 = NULL;
```



```
int a = 10;
```

```
int* p1 = &a;
```

```
int* p2 = NULL;
```

```
p2 = p1;
```

```
*p2 = 20;
```

Memória Ram (fictícia)

H000	####	
...	####	
S012	S000	p2
S004	S000	p1
S000	10	a



```
&a = S000; a = 10;  
&p1 = S004; p1 = S000; *p1 = 10;  
&p2 = S012; p2 = S000; *p2 = 10;
```



```
int a = 10;  
  
int* p1 = &a;  
int* p2 = NULL;  
  
p2 = p1;  
*p2 = 20;
```

Memória Ram (fictícia)

H000	####	
...	####	
S012	S000	p2
S004	S000	p1
S000	20	a



&a = S000; a = 20;
&p1 = S004; p1 = S000; *p1 = 20;
&p2 = S012; p2 = S000; *p2 = 20;



Exercício 1: Operações com Ponteiros

Escreva uma função que recebe dois ponteiros para valores inteiros e troca o valor dessas variáveis

Vetores



Obrigado,
Gustavo do passado
pelos slides e desenhos

Vetores: Definição e Sintaxe

Um **vetor** ou **array** consiste em uma **estrutura de dados linear** que armazena uma sequência de elementos de **tamanho fixo**, todos do **mesmo tipo**, em **posições de memória contíguas**.

The diagram illustrates the syntax of a vector declaration. It shows a general form at the top and a specific example at the bottom, with arrows indicating the correspondence between the two.

```
tipo_dado  nome_vetor[tamanho_vetor];
```

↓ ↓ ↓

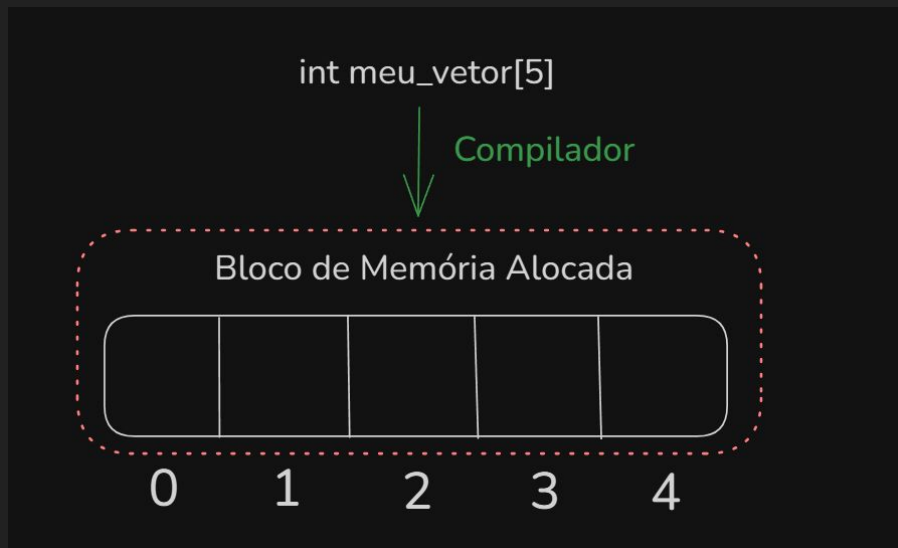
```
int  meu_vetor[5];
```

The arrows show that 'tipo_dado' maps to 'int', 'nome_vetor' maps to 'meu_vetor', and 'tamanho_vetor' maps to '5'.

Vetores: Definição e Sintaxe

Quando declaramos um array em C, o **compilador aloca** um bloco de memória do tamanho **necessário** e **especificado** durante a declaração da variável.

Cada item de um vetor é **indexado** de **0** à **n - 1**, sendo n o tamanho do vetor.



Vetores: Definição e Sintaxe

A variável que acabamos de criar na realidade consiste em um **ponteiro** que aponta para o primeiro elemento do vetor.



```
1  #include <stdio.h>
2
3  int main() {
4      int meu_vetor[5];
5      printf("%p\n", meu_vetor);
6  }
7
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

gustavo in ~/mini-curso-c-aula05
→ g++ exemplo.c && ./a.out
0x7ffc1d7a0a30

Vetores: Definição e Sintaxe (Prática)

Prática 01:

Crie um vetor e exiba o **endereço** que foi reservado para essa variável.



Vetores: Acessando elementos de um vetor

Podemos acessar elementos de um vetor a partir da seguinte notação:

<code>meu_vetor[0]</code>	→	Primeiro Elemento (Posição 0)
<code>meu_vetor[1]</code>	→	Segundo Elemento (Posição 1)
...		
<code>meu_vetor[i]</code>	→	i-ésimo Elemento (Posição i)
...		
<code>meu_vetor[n - 1]</code>	→	Último Elemento (Posição n - 1)

Também é possível fazer esse acesso a partir de **Aritmética de Ponteiros** 🕶️

Vetores: Acessando elementos de um vetor

Pergunta: Em um vetor de tamanho n , o que acontece se tentarmos acessar um elemento que está fora do intervalo $[0, n - 1]$?

Resposta Inocente: Vai “levantar” um **Index Out of Bounds**? **NÃO**

O programa vai tentar acessar uma **área de memória não destinada ao vetor**.

Esse endereço acessado pode ser:

- De uma **outra variável do mesmo programa**.
- Uma **área protegida pelo sistema operacional** → Um **segmentation fault** pode ser levantado.

Vetores: Acessando elementos de um vetor

O valor lido ou escrito nessa posição será **imprevisível** (pode ser **lixo de memória**, **zero** ou até **sobrescrever dados importantes**).

```
1  #include <stdio.h>
2
3  int main() {
4      int meu_vetor[5];
5      printf("%d ", meu_vetor[0]);
6      printf("%d ", meu_vetor[1]);
7      printf("%d ", meu_vetor[2]);
8      printf("%d ", meu_vetor[3]);
9      printf("%d ", meu_vetor[4]);
10     printf("%d\n", meu_vetor[5]);
11 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

gustavo in ~/mini-curso-c-aula05

→ g++ exemplo.c && ./a.out

0 0 -1869587728 28946 840065552 32765

gustavo in ~/mini-curso-c-aula05

→ g++ exemplo.c && ./a.out

0 0 1974467312 30615 -1229745056 32764



Vetores: Acessando elementos de um vetor

Prática 02:

Como demonstrado no slide anterior, crie um vetor de tamanho 5 sem inicializá-lo e exiba os valores armazenados nas posições de 0 a 5 (inclusive).



Vetores: Inicializando um vetor

Como visto no experimento anterior, os elementos acessados do vetor tinham **valores aleatórios**.

Em C, há diversas formas de **inicializar** um vetor no instante de sua declaração, definindo os valores que ele armazenará.

→ Inicialização Completa



```
1  int v[5] = {1, 2, 3, 4, 5};
```

Vetores: Inicializando um vetor

→ Inicialização Parcial

Se você fornecer menos valores do que o tamanho declarado, o compilador completa o restante com zeros



```
1  int v[5] = {1, 2, 3};
```

Vetores: Inicializando um vetor

→ Deixar o compilador calcular o tamanho

Se você não especificar o tamanho, mas inicializar com valores, o compilador conta automaticamente.



```
1  int v[] = {1, 2, 3, 4, 5};
```

Vetores: Inicializando um vetor

→ Inicialização com atalho

Caso você passe apenas `{X}`, todos os elementos do vetor serão inicializados com o valor `X`.

Por exemplo, podemos inicializar um vetor com zeros rapidamente.



```
1  int v[5] = {0};
```

Vetores: Percorrendo um vetor

Para percorrer um vetor em C utilizamos o conceito de **loops** (laços).

```
1  #include <stdio.h>
2
3  int main() {
4      int arr[5] = {2, 4, 8, 12, 16};
5
6      printf("Exibindo elementos do Vetor\n");
7      for(int i = 0; i < 5; i++){
8          printf("%d ", arr[i]);
9      }
10     printf("\n");
11     printf("Exibindo elementos do Vetor (REVERSO)\n");
12     for(int i = 4; i >= 0; i--){
13         printf("%d ", arr[i]);
14     }
15     return 0;
16 }
```

CONDIÇÃO DE PARADA

$i = \{0, 1, 2, 3, 4, \boxed{5}\}$

CONDIÇÃO DE PARADA

$i = \{4, 3, 2, 1, 0, \boxed{-1}\}$

Vetores: Percorrendo um vetor

Note que ao criar nosso laço para percorrer podemos dividir o processo em **sub-tarefas**:

1. Enumerar uma **lista de índices** queremos acessar.
2. Identificar o índice **inicial** e **final**.
3. Ajustar a **condição de parada**.
 - a. Lista de índices Crescente → **<** ou **<=**
 - b. Lista de índices Decrescente → **>** ou **>=**
4. Definir o **Incremento/Decremento**.
 - a. Lista de índices Crescente → **i++** ou **i += x**
 - b. Lista de índices Decrescente → **i--** ou **i -= x**

```
for(inicialização ; condição ; incremento  
    de parada ; decremento )
```

Exercício 2: Operações com Vetor

Escreva um programa em C que realize as seguintes operações com um vetor de 5 números inteiros:

1. **Leia** os 5 valores inteiros digitados pelo usuário e armazene-os em um vetor.
2. **Exiba** todos os números digitados na mesma ordem em que foram inseridos.
3. Calcule e mostre a **soma e a média aritmética** dos elementos do vetor.

Solução 2: Operações com Vetor



```
1  #include <stdio.h>
2
3  int main() {
4      int v[5];
5      // Q1 - Leitura do vetor
6      for (int i = 0; i < 5; i++)
7          scanf("%d", &v[i]);
8
9      // Q2 - Exibir o vetor
10     for (int i = 0; i < 5; i++)
11         printf("%d ", v[i]);
12     printf("\n");
```



```
1      // Q3 - Calcular soma e media
2      long long soma = 0LL;
3      for (int i = 0; i < 5; i++) soma += v[i];
4
5      double media = (double) soma / 5;
6
7      printf("Soma = %lld | Media = %lf\n", soma, media);
8
9      return 0;
10 }
```

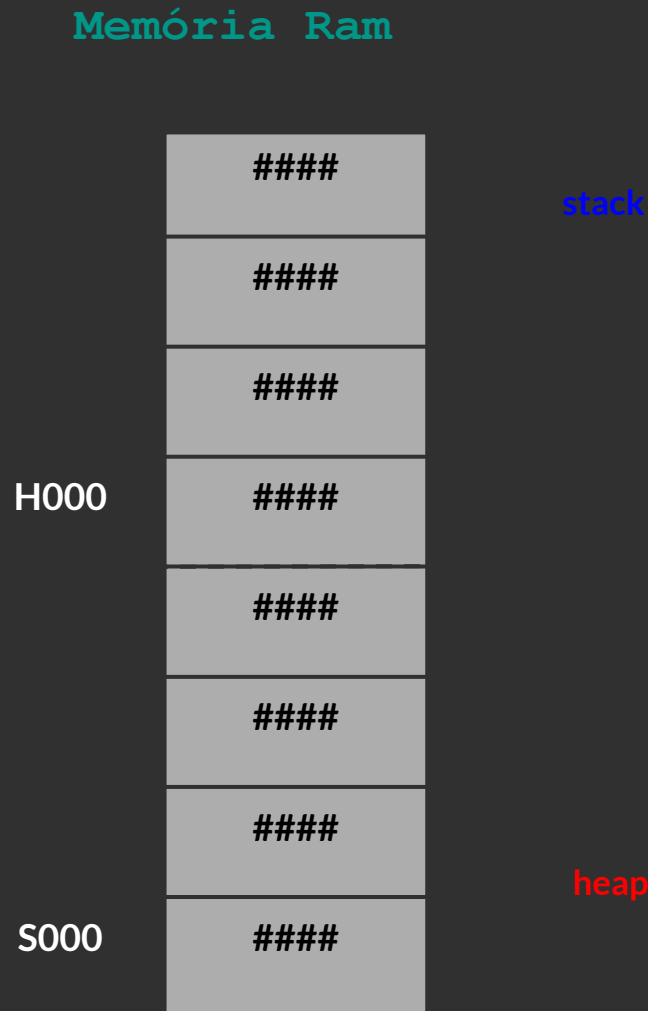
Alocação Dinâmica



Obrigado, Calumbi pelos
slides e desenhos

Alocação de memória dinâmica

Alocando memória em tempo de execução



Memória estática

- O espaço é reservado no início da execução do programa
- Cada variável tem seu endereço e tamanho fixado
- São alocadas na Stack da memória ram
- A liberação da memória é feita automaticamente após a finalização do programa

Memória dinâmica

- O espaço é reservado durante a execução do programa
- Pode ser criada ou eliminada durante a execução do programa
- São alocadas na Heap da memória ram
- A liberação da memória é feita manualmente pelo programador

Alocando memória com 'malloc'

Vamos entender como
funciona



```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int N = 10;
    int* vetor;
    vetor = (int*) malloc(N * sizeof(int));

    vetor[0] = 10;
    vetor[1] = 90;
    vetor[9] = 45;

    free(vetor);
    vetor = NULL;

    return 0;
}
```



```
#include <stdio.h>
#include <stdlib.h>

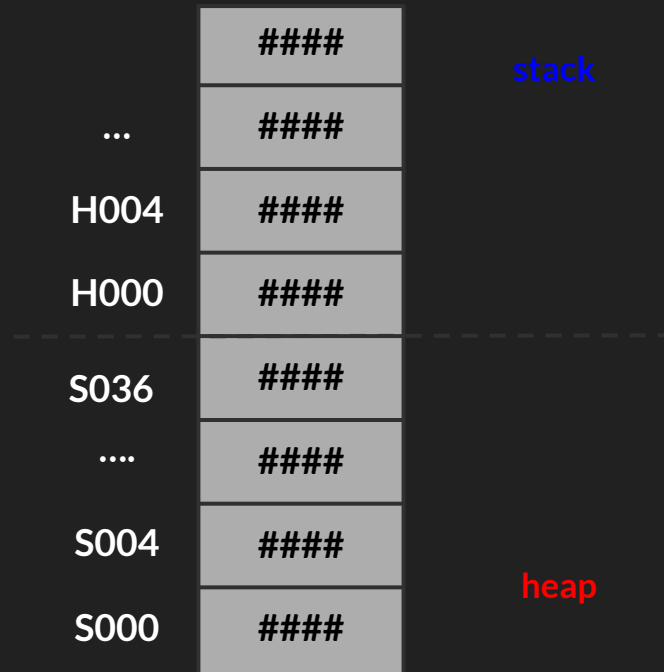
int main(void)
{
    int N = 10;
    int* vetor;
    vetor = (int*) malloc(N * sizeof(int));

    vetor[0] = 10;
    vetor[1] = 90;
    vetor[9] = 45;

    free(vetor);
    vetor = NULL;

    return 0;
}
```

Memória Ram





```
#include <stdio.h>
#include <stdlib.h>
```

```
int main(void)
{
    int N = 10;
    int* vetor;
    vetor = (int*) malloc(N * sizeof(int));

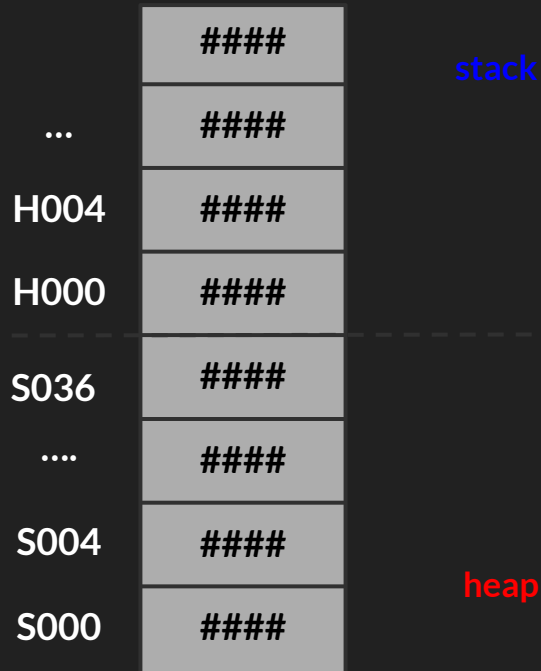
    vetor[0] = 10;
    vetor[1] = 90;
    vetor[9] = 45;

    free(vetor);
    vetor = NULL;

    return 0;
}
```



Memória Ram





```
#include <stdio.h>
#include <stdlib.h>
```

```
int main(void)
{
```

```
    int N = 10;
```

```
    int* vetor;
```

```
    vetor = (int*) malloc(N * sizeof(int));
```

```
    vetor[0] = 10;
```

```
    vetor[1] = 90;
```

```
    vetor[9] = 45;
```

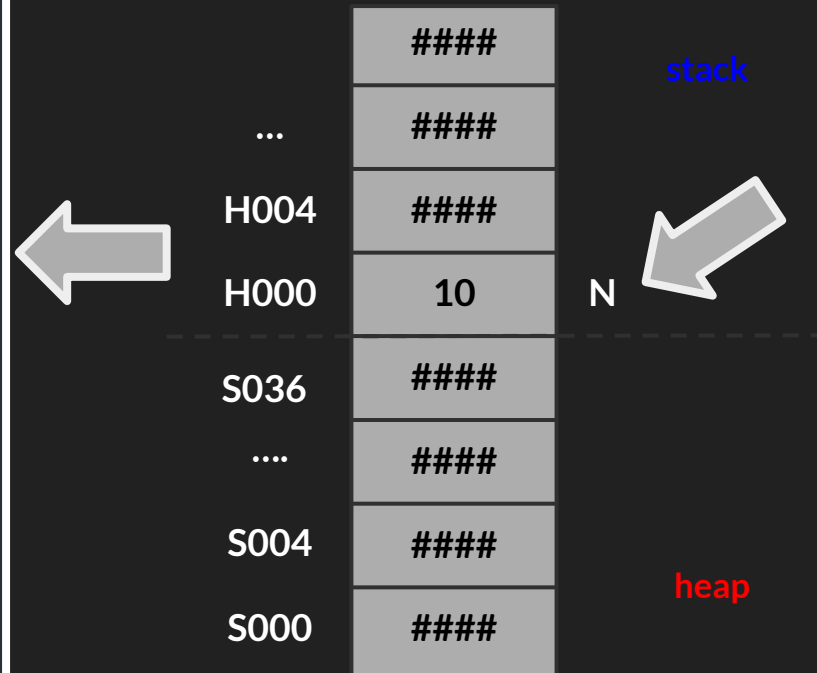
```
    free(vetor);
```

```
    vetor = NULL;
```

```
    return 0;
```

```
}
```

Memória Ram





```
#include <stdio.h>
#include <stdlib.h>

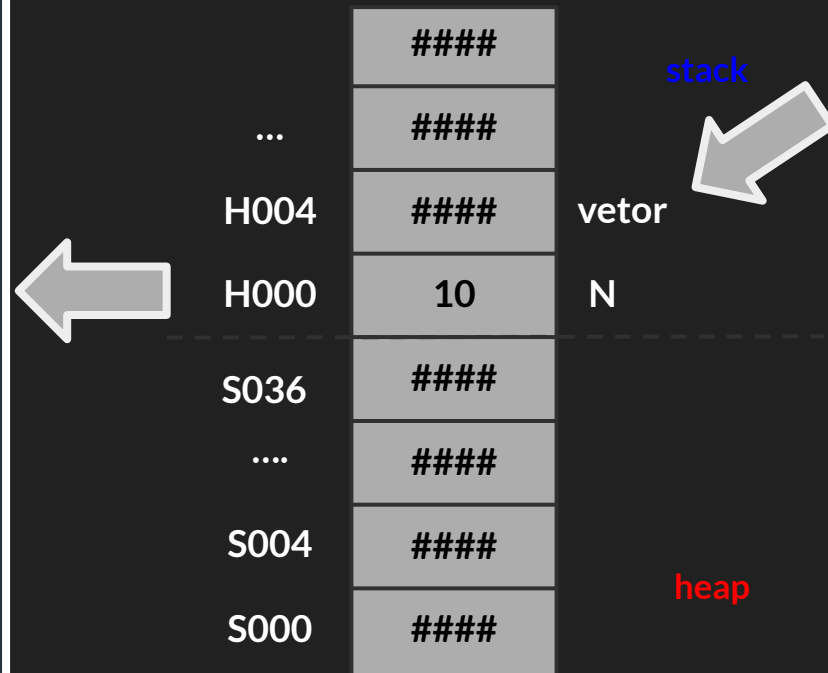
int main(void)
{
    int N = 10;
    int* vetor;
    vetor = (int*) malloc(N * sizeof(int));

    vetor[0] = 10;
    vetor[1] = 90;
    vetor[9] = 45;

    free(vetor);
    vetor = NULL;

    return 0;
}
```

Memória Ram





```
#include <stdio.h>
#include <stdlib.h>
```

```
int main(void)
{
```

```
    int N = 10;
    int* vetor;
```

```
    vetor = (int*) malloc(N * sizeof(int));
```

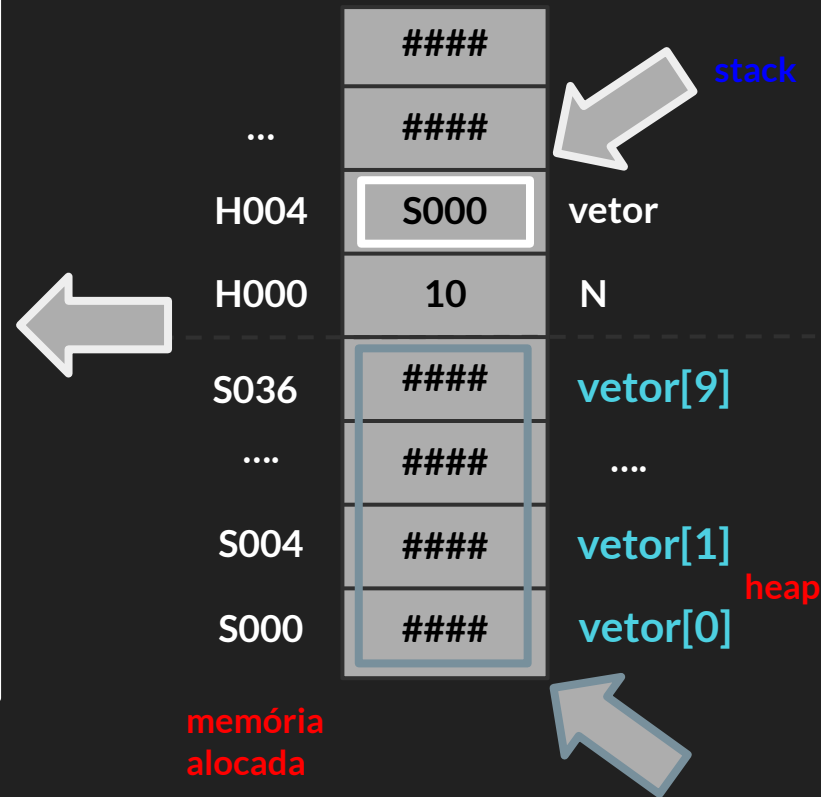
```
    vetor[0] = 10;
    vetor[1] = 90;
    vetor[9] = 45;
```

```
    free(vetor);
    vetor = NULL;
```

```
    return 0;
```

```
}
```

Memória Ram



**** dizemos que o ponteiro 'vetor' aponta para S000**



```
#include <stdio.h>
#include <stdlib.h>
```

```
int main(void)
{
```

```
    int N = 10;
    int* vetor;
```

```
    vetor = (int*) malloc(N * sizeof(int));
```

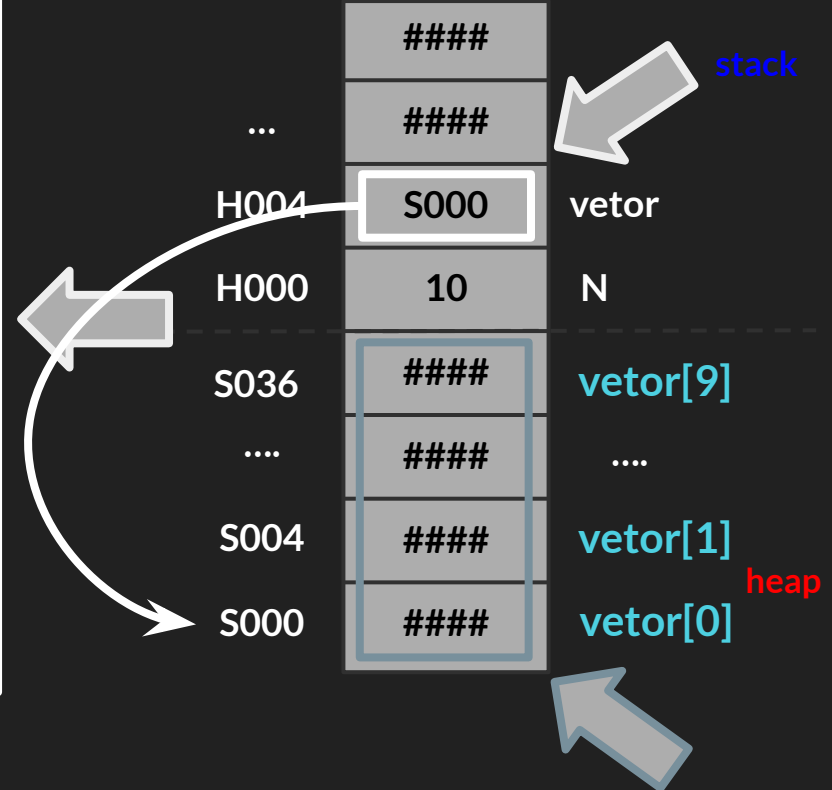
```
    vetor[0] = 10;
    vetor[1] = 90;
    vetor[9] = 45;
```

```
    free(vetor);
    vetor = NULL;
```

```
    return 0;
```

```
}
```

Memória Ram





```
#include <stdio.h>
#include <stdlib.h>

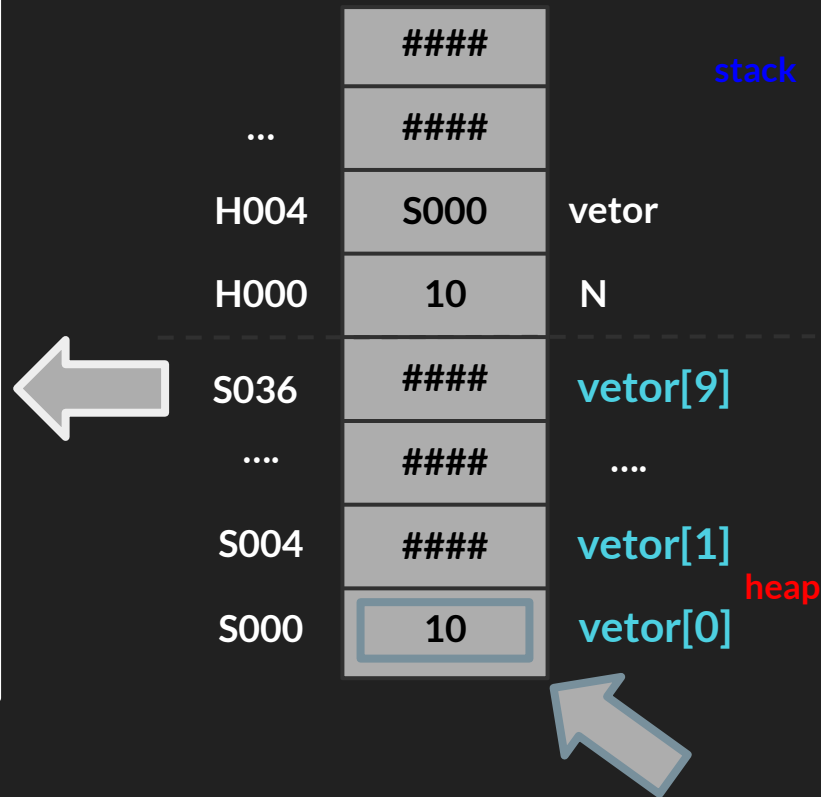
int main(void)
{
    int N = 10;
    int* vetor;
    vetor = (int*) malloc(N * sizeof(int));

    vetor[0] = 10;
    vetor[1] = 90;
    vetor[9] = 45;

    free(vetor);
    vetor = NULL;

    return 0;
}
```

Memória Ram





```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int N = 10;
    int* vetor;
    vetor = (int*) malloc(N * sizeof(int));

    vetor[0] = 10;
    vetor[1] = 90;
    vetor[9] = 45;

    free(vetor);
    vetor = NULL;

    return 0;
}
```

Memória Ram

	####	
...	####	stack
H004	S000	vetor
H000	10	N
S036	####	vetor[9]
...	####	...
S004	90	vetor[1]
S000	10	vetor[0]



heap



```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int N = 10;
    int* vetor;
    vetor = (int*) malloc(N * sizeof(int));

    vetor[0] = 10;
    vetor[1] = 90;
    vetor[9] = 45;

    free(vetor);
    vetor = NULL;

    return 0;
}
```

Memória Ram

	####	
...	####	
H004	S000	vetor
H000	10	N
S036	45	vetor[9]
...	####	
S004	90	vetor[1]
S000	10	vetor[0]



stack

heap



```
#include <stdio.h>
#include <stdlib.h>

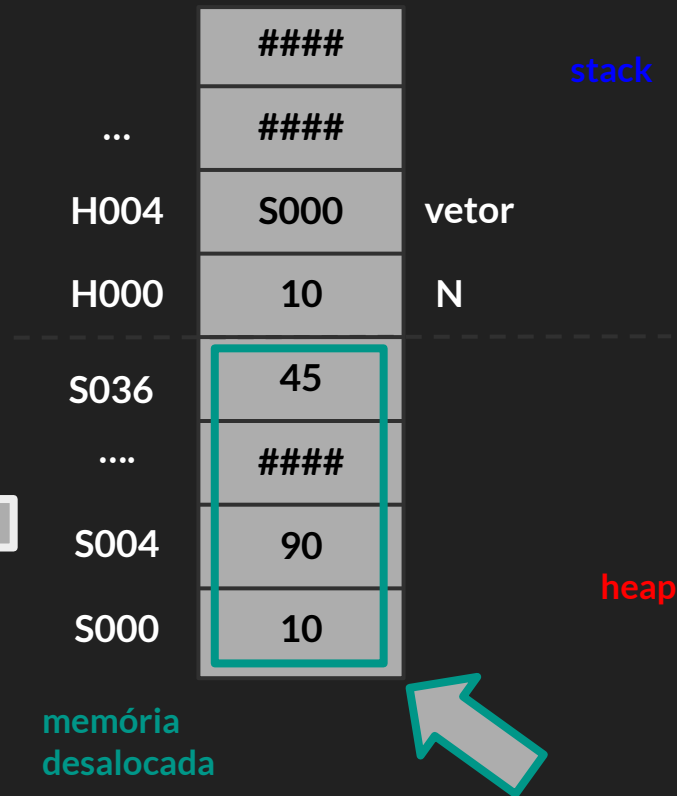
int main(void)
{
    int N = 10;
    int* vetor;
    vetor = (int*) malloc(N * sizeof(int));

    vetor[0] = 10;
    vetor[1] = 90;
    vetor[9] = 45;

    free(vetor);
    vetor = NULL;

    return 0;
}
```

Memória Ram





```
#include <stdio.h>
#include <stdlib.h>

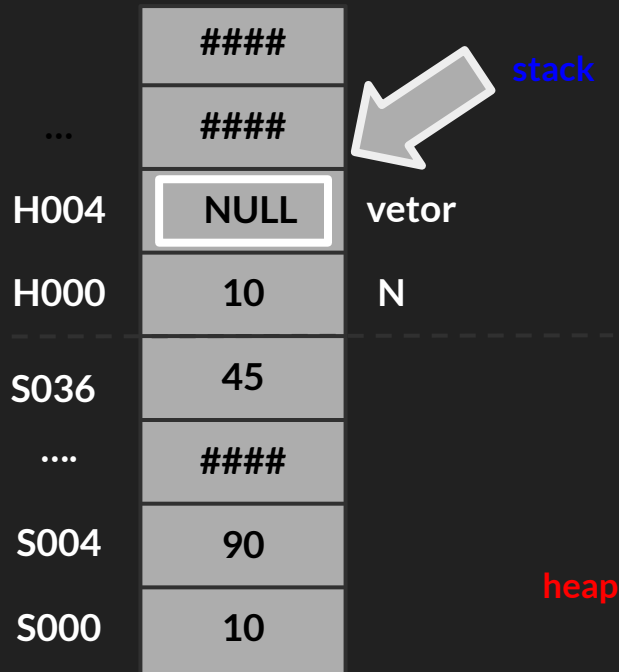
int main(void)
{
    int N = 10;
    int* vetor;
    vetor = (int*) malloc(N * sizeof(int));

    vetor[0] = 10;
    vetor[1] = 90;
    vetor[9] = 45;

    free(vetor);
    vetor = NULL;

    return 0;
}
```

Memória Ram



Alocando memória com 'calloc'

Faz o mesmo que
malloc, mas os bits são
inicializados com zero.



```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int N = 10;
    int* vetor;
    vetor = (int*) calloc(N, sizeof(int));

    // codigo

    free(vetor);
    vetor = NULL;

    return 0;
}
```

Exercício 3: Alocação dinâmica

Escreva um programa em C que:

1. Leia um número inteiro N informado pelo usuário.
2. Aloque dinamicamente uma matriz $N \times N$;
3. Preencha a matriz;
4. Libere a memória da matriz;

Exercício 4: Alocação dinâmica (Extra 🔥)

Escreva um programa em C que:

1. Leia um número inteiro N informado pelo usuário.
2. Aloque dinamicamente um vetor de N inteiros.
3. Preencha o vetor com scanf e utilizando aritmética de ponteiros.

*dica: ao alocar dinamicamente um vetor, utilizando:

`tipo * point = (tipo *) malloc();`

a variável *point* é um endereço de memória 🐼

Aprendam com a monitoria



Em breve... 👁👁



Dúvidas no Discord!

Vamos aprender juntos!



**Dúvidas e
Feedbacks?**

Obrigado pela atenção

